Evaluating Coding Behavior in Software Development Processes: A Process Mining Approach

Pasquale Ardimento
University of Bari Aldo Moro
Bari, Italy
pasquale.ardimento@uniba.it

Mario Luca Bernardi
Giustino Fortunato University
Benevento, Italy
m.bernardi@unifortunato.eu

Marta Cimitile
Unitelma Sapienza
Rome, Italy
marta.cimitile@unitelmasapienza.it

Fabrizio Maria Maggi
University of Tartu
Tartu, Estonia
f.m.maggi@ut.ee

***Abstract*—Process mining is a family of techniques that aim at analyzing business process execution data recorded in event logs. Conformance checking is a branch of this discipline embracing approaches for verifying whether the behavior of a process, as recorded in a log, is in line with some expected behavior provided in the form of a process model. In the literature, process mining techniques have already been used to study software development processes starting from logs derived from version management systems or from document management systems. In this paper, we use conformance checking to test coding behaviors starting from event logs generated from IDE usage. Understanding how developers carry out coding activities and what hurdles they usually face should provide useful tips for improving and supporting software development processes. In particular, through conformance checking, we can compare different process executions, and identify behavioral similarities and differences. In our experimentation, we evaluated the activities performed by 40 novice developers performing coding activities in 5 development sessions. We assessed the developers to distinguish the ones obtaining the best performance. We then compared the behavior extracted from this group of developers with the others. The results show different IDE usage patterns for developers with different skills and performance.**

***Index Terms*—Software Development Process, Process Mining, Conformance Checking, IDE Logging**

I. Introduction

Enhancing the quality of the coding practices of students and novice developers has historically been a major interest in software engineering and computing education domains. This could be achieved through the adoption of software development principles, practices, and tools — like integrated development environments (IDEs) — but can also be obtained through the application of learning analytics approaches (e.g., process mining) to study the behavior of different individuals when developing software artifacts.

The significant performance gap between high- and low-skilled developers [7] hints that the coding capabilities of the latter can highly benefit from a deep com hension of the workflows describing how the former face software development. However, the comprehension of software development processes is not a simple task. By nature, software development is profoundly iterative and characterized by a very loose ordering of activities [8]. Developers, starting from requirements, build a model of the software artifact to be developed by extracting information coming from different sources and synthesize a suitable design solution [3]. This involves the hierarchical breakdown of the artifact into smaller components, and, in turn, the choice or development of algorithms to implement such components [16]. This is a complex process in which attitude, knowledge, and experience of developers have a great impact on their behavior when coding. The awareness of the above-discussed critical issues has fueled the interest of researchers in the application of process mining techniques to understand the behavior of developers in software development processes, as many studies on the subject testify [2], [19], [10], [14]. These techniques provide a clear picture of the behavior of the developers by analyzing the data recorded during the development process. In addition, most of the existing process mining approaches allow the analysts to check the logs at different levels of granularity and to filter out unfrequent/noisy behaviors thus making the process representations more understandable.

In this work, we mainly focus on the use of an approach based on conformance checking, a branch of process mining aiming at comparing process executions recorded in event logs with some expected behavior represented as a process model.

In particular, we provide the following contributions:

- we illustrate an environment, which extends an existing Eclipse plug-in (Fluorite[1]) to produce event logs that can

[1] http://www.cs.cmu.edu/~fluorite/index.html

be analyzed using process mining techniques.

- we present the results of an exploratory study in which event logs of 5 development sessions, involving 40 students of an advanced Java course, were analyzed with a process mining tool based on Directly Follows Graphs (DFGs) [11] to compare different coding patterns. Specifically, the development processes of two groups respectively of high and low-performing participants are mined to extract a workflow representing good development practices. These reference models are then used to check the conformance with the development processes of the other participants to understand the main behavioral differences.

The rest of the paper is structured as follows. Section II discusses the background. In Section III, the proposed environment is introduced, whereas, in Section IV, an evaluation showing the use of proposed methodology in a real domain is presented and discussed. Finally, Section V discusses related work and Section VI provides conclusions.

II. Background

In this section, we first give an overview on process mining and, then, we introduce a technique for DFG mining we use in our experiments.

A. Process Mining

Process mining is concerned with the analysis of collections of event records produced during the execution of a business process. Such event records represent events signaling the start, end, abortion or other relevant state change of a process or an activity therein, or any other event of relevance to the execution of a process such as the allocation or de-allocation of a process worker to a task, the receipt or dispatching of a message to a process participant, and so on. In general, the main input of a process mining technique is an *event log*, which is a collection of event records relevant to a given process. An event log is generally structured as a set of traces, where each trace consists of the sequence of events produced by one execution of the process (a case). As a minimum, an event record contains an identifier of the case of the process to which the event refers, a timestamp, and possibly a number of additional attributes.

Based on such event logs, process mining techniques extract useful information that allows analysts to gain insights into the process and to formulate and validate hypotheses about the process. The output of process mining can be manifold, ranging from a complete model of the process, to a description of the main (frequent) paths of the process or its deviations, or a diagnosis of the reasons for deviations in a process. Process mining techniques can be broadly classified into descriptive techniques and predictive techniques. Descriptive techniques aim at providing insights about the process as it is or as it has been observed in the logs. Predictive techniques aim at predicting how a process as a whole or a specific case of the process will behave in future under certain hypotheses or conditions.

B. DFG Mining

This paper uses the DFG mining technique described in [11]. The approach is based on the extraction of DFGs from logs. Precisely, a DFG represents activities as rectangles and connects two activities if one of them directly follows the other. Moreover, each edge has a weight indicating the number of its occurrences in the log. The extraction of a DFG from an event log is performed by using the Inductive Miner directly-follows (IMd) framework [11]. The IMd is based on the assumption that the model is representable by a process tree [4] having specific characteristics (has no duplicate activities, contains no silent activities and where all loop bodies contain no activity that is both a start and end activity of that loop body) and ensures that the discovered model is behaviorally correct and with high fitness [12].

Moreover, in [11], the authors also propose a conformance checking approach to compare a log with a DFG. The approach takes as input an event log and a DFG. From both the log and the DFG a deterministic finite automata (DFA) is constructed. The comparison between the DFA extracted from the log and the DFA extracted from the DFG is obtained evaluating a process model precision metric (PMP). PMP is defined as the part of behavior in the DFG that is also in the log [11]. In our experimentation, we use this tool to compare the behavior of high- and low-performing developers.

III. Approach

The proposed approach aims at defining a monitored development environment where all the coding activities are captured in event logs. Moreover, the proposed environment also allows evaluating the conformance of the captured logs with respect to a process model. In this work, we mine the reference process models by a set of traces extracted during the high and low-profile developers' coding sessions (we also use the model discovered from the log containing the coding sessions of the low-performing developers as reference model to have a complete picture of the differences between the coding behaviors of high- and low-performing developers).

Fig. 1 gives an overview of the proposed mining environment based on IDE instrumentation. The figure describes three main components: the Log Processor, the DFG Miner and the Conformance Checker. The Log Processor is the component that captures the human-computer interaction (HCI) events (they are the stream of activities performed by the developers when they interact with the development environment) and stores the collected logs in a development sessions logs repository. All the collected logs need to be refined in order to apply process mining. Specifically, the following steps are executed:

1) interactions that are unrelated with projects in the developer workspace are removed;
2) inconsistencies in the data are detected and corrected;
3) the low-level logs are converted into the XES[2] event log model;

[2]http://www.xes-standard.org

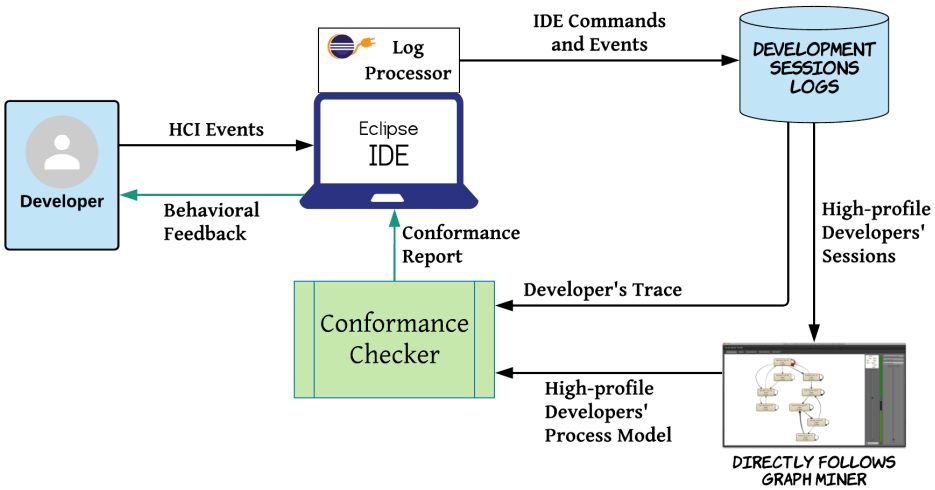

Fig. 1. An overview of the proposed framework.

4) low-level and high-level event logs are integrated.

The XES logs corresponding to the low and high-profile developers are then mined using the DFG Miner to discover the models (representing the coding behavior of the low and high-profile developers). The traces coming from new developers can be analyzed using the Conformance Checker in order to compare the conformance of the coding process executed by these developers with respect the low-profile and the high-profile developers' process model.

A. The Log Processor

The Log Processor is an extension of the Fluorite plug-in, an open-source instrumentation plug-in, capable of recording low-level developers' interactions with the IDE without interrupting the coding activities. By design, it can only track low-level interactions that happen in the editor (e.g., mainly keyboard key presses, shortcuts, mouse movements and gestures). Although the Fluorite plug-in provides useful information on how developers interact with the editor, the captured events are not suitable to enable the mining of the coding processes. The limitations of the Fluorite plug-in for our goals are twofold: (*i*) low-level events need additional contextual information and (*ii*) high-level events (e.g., all the commands issued at IDE level, such as create or open a file, close a project, open a view, reset perspective, etc.) are substantially missing. The Log Processor extends the Fluorite plug-in fixing both these limitations.

1) Low-Level events augmentation: The main Fluorite plug-in abstraction is the DocumentChangeEvent class that provides information on almost all the low-level events happening during development. However, the Fluorite plug-in does not have the notion of a case id. Therefore, we decided to add a synthetic id to group events in cases. Specifically, the case id refers to a single development session performed by a single developer. Another limitation to overcome was due to the nature of Fluorite with respect to the Eclipse resource framework: the collected events had no relation with the involved resources (including file resources). To address this, an explicit FileSwitchEvent linked to a FileElement was introduced and integrated into the Eclipse environment. The modifications performed to the Fluorite low-level events model are shown in Fig. 3: the classes with gray background are the pre-existing concepts provided by Fluorite, whereas the classes with the blue background are the implemented additions. With these extensions, it is possible to link Fluorite events to developers' actions on resources in the IDE.

2) High-level developer events: The second limitation of the Fluorite plug-in is related to the event abstraction level: the plug-in is able to capture low-level events but ignores almost all high-level events and specifically ignores the IDE commands. Therefore, in the Log Processor, we had to expand the captured event model in order to produce process logs that contain developers behavior at IDE level. An excerpt of the proposed event model is shown in the UML class diagram in Fig. 2. The root of the high-level event hierarchy is the IDE_Event class that models a generic event at IDE level (i.e., an action or command issued by the developer in the IDE). Each subclass of IDE_Event models a specific kind of interaction and contains contextual information (e.g., the involved resources and their possible state changes). The four most important sub-classes modeling the main aspects of developer interactions are reported below:

- **IDECommandEvent**: modeling all global commands issued to the IDE (e.g., open a view, switch a perspective or accessing a resource like a file);
- **EditorCommandEvent**: modeling all activities happening in the editor (e.g., cut and paste commands, text selection and modification, and all actions regarding code writing);
- **DebuggingEvent**: modeling all debugging activities (e.g., breakpoints definition, watches on variables and their inspection at runtime, and debug profiles definitions);

- **RefactoringEvent**: modeling all the commands related to refactoring activities (e.g., selecting a refactoring among the ones available, instructing the IDE on how to perform it, and launch its execution).

Each of these sub-classes has, in turn, concrete sub-classes describing specific events happening during the interactions.

B. Directly Follows Graph Miner and Conformance Checker Components

The low and high profile developers' sessions events logs are mined by the DFG Miner [11] included in the ProM toolkit [9]. This miner is used to build a process model representing the behavior of all the developers having the worse and the best performance in coding tasks. In particular, the miner takes as input the logs of the low and high profile developers' sessions captured by the Log Processor and creates an appropriate representation of the development processes expressed as DFGs (as defined in Section II). Finally, conformance checking is performed by using the Visualize deviations on DirectlyFollowsModel ProM plug-in [11]. It takes as input a DFG and a log and produces a graphical representation of the deviations between the DFG and the log. Fig. 1 shows how the checker is used in the proposed environment: it takes as input the DFG representing the low-profile and high-profile developers' behavior and compares it with new developers that need to be evaluated. The obtained result is a conformance report that is used by the IDE to produce behavioral feedback.

IV. Experimentation

This section explains the exploratory study we conducted to evaluate our framework.

A. Design and Setup

The study involved 40 students from an advanced Java course held in the Department of Computer Science at the University of Bari Aldo Moro (Italy). All of them had previous experience in development with C and Java. Their age ranged from 19 to 21, and all of them had experience using Eclipse in a previous basic Java course. Participants took part to 5 development sessions. Each session consisted in a fairly simple programming task taking ≈ 2 hours for completion. The participants, to perform the development, had access to a personal copy of the Eclipse environment equipped with the log processor. At the end of each session, the students delivered a package produced by the Eclipse environment containing the logs collected during the development session, the screen-shots sampled at 2Hz, and the code written for the given assignment. Information about the author was automatically injected in the source code, as comments, to mark the author's identity. The Eclipse environment encrypts the package with a private key known only to instructors: this helps to prevent from participants delivering fake logs or importing source code from other participants. All the logs extracted from the student packages were collected and analyzed with the DFG Miner.

The code contained in each package was evaluated by the teacher of the course. The evaluation assigned a grade ranging from 17 (insufficient performance) to 30 (excellent performance) to each student. In all the development sessions, the participants were required to implement a small software artifact using Java. Table I reports (columns from two to six) for each student (the student id is in the first column) and for each exercise session (Ex_1, Ex_2, Ex_3, Ex_4, Ex_5) the corresponding grade. The students were profiled (column eight) on the base of the average of their grades (column seven) as "Low" and "High". In particular, students have a "Low" profile if they have an average grade lower than 25. Students with "High" profile have an average grade that is greater than or equal to 25. The table shows that, on the base of the student code evaluation, we have a log of 100 traces (5 exercise sessions for twenty students) describing the joint development behaviors of students with "High" profile and a log of 100 traces describing the joint development behaviors of students with "Low" profile. Each trace represents a single development session of 2 hours in which a single student carries out the assignment. For both performance profiles (low and high), we randomly divided the log into a training set composed of 60 traces, a validation set composed of 20 traces, and a test set composed of 20 traces. The training sets have been used for extracting two DFGs one representing the behavior of high-profile students and one representing the behavior of

TABLE I
GRADES GIVEN TO STUDENTS BY THE TEACHER

Student ID	Evaluations by Assignment Ex1	Ex2	Ex3	Ex4	Ex5	Average Evaluation	Profile
1	28	26	27	27	28	27,2	High
3	24	28	26	27	29	26,8	
5	27	28	27	26	25	26,6	
7	22	30	28	27	28	27	
9	27	27	26	26	26	26,4	
11	25	30	25	25	26	26,2	
13	26	28	25	28	28	27	
15	25	30	25	28	25	26,6	
17	20	30	27	27	26	26	
19	26	25	27	26	26	26	
21	20	30	28	27	25	26	
23	26	28	26	25	28	26,6	
25	20	30	28	26	28	26,4	
27	26	28	28	26	26	26,8	
29	30	18	25	28	30	26,2	
31	28	30	28	30	30	29,2	
33	30	25	30	28	30	28,6	
35	27	29	28	27	28	27,8	
37	30	30	27	28	30	29	
39	27	25	28	27	26	26,6	
2	19	19	17	17	19	18,2	Low
4	18	20	18	18	18	18,4	
6	18	17	18	18	19	18	
8	26	18	26	20	26	23,2	
10	25	23	24	27	25	24,8	
12	23	24	22	25	23	23,4	
14	21	24	22	26	24	23,4	
16	23	23	22	22	23	22,6	
18	20	18	20	20	18	19,2	
20	21	21	22	22	23	21,8	
22	20	18	20	20	20	19,6	
24	22	18	22	20	22	20,8	
26	21	22	20	22	21	21,2	
28	24	23	23	26	24	24	
30	18	20	18	18	18	18,4	
32	20	18	20	19	20	19,4	
34	22	20	22	22	22	21,6	
36	21	21	20	21	23	21,2	
38	22	25	18	20	20	21	
40	26	24	24	21	23	23,6	

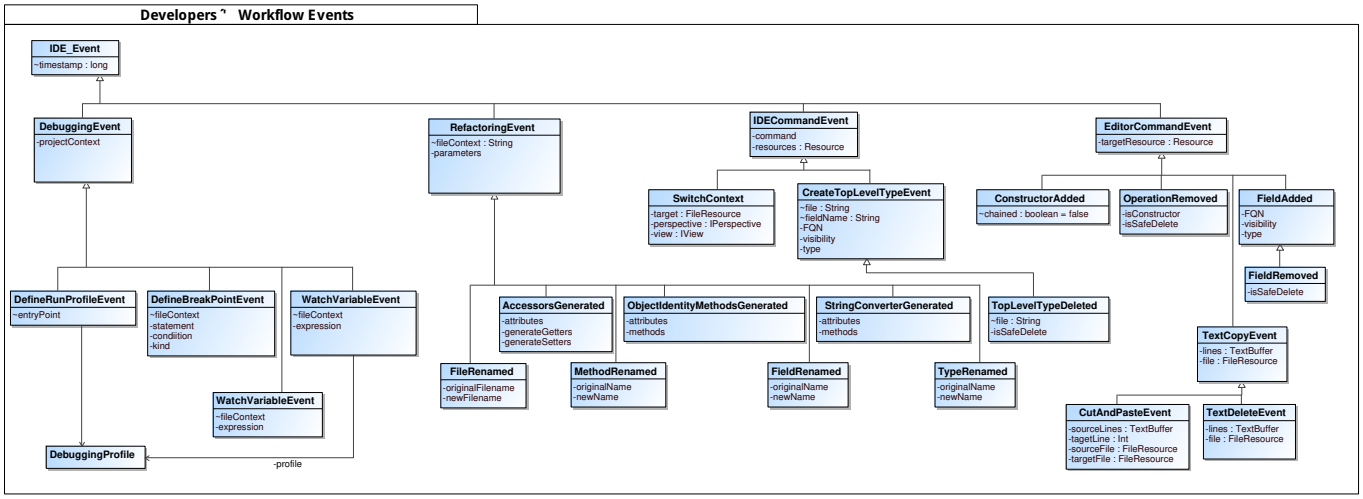

Fig. 2. The high-level event model captured by the proposed IDE environment.

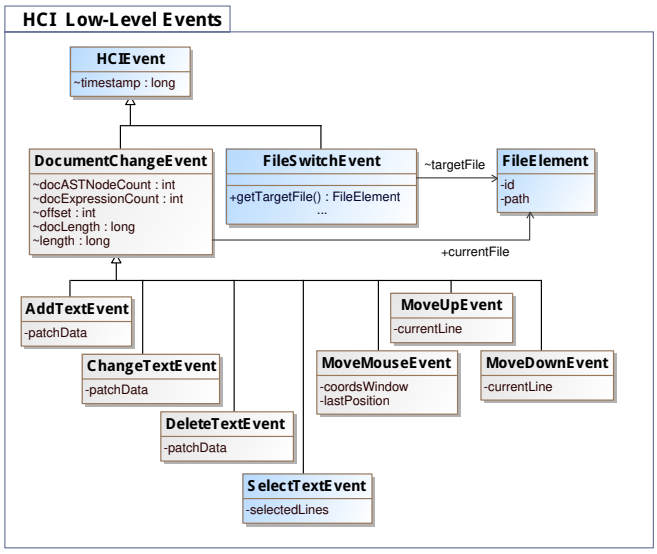

Fig. 3. The low-level event model of the Fluorite plug-in (in gray) and the implemented extensions (in blue).

low-profile students. For each profile, we then used the union of the two validation sets to identify the PMP threshold to be used to discriminate between high-profile and low-profile developers. Finally, we used the union of the two test sets to evaluate the accuracy of the approach in the classification of the profile of a new developer as high or low.

B. Discussion of the Results

The overall process generated by mining the 60 traces describing the students with "High" profile during the 5 development sessions is reported in Fig. 4. Similarly, Fig. 5 describes the mined process for students with low performance. The figures show process activities performed by students (rectangle shapes) and the control flow relations between them (edges connecting two activities). The figures also report, in the model for each of the two profiles, the differences with the behavior present in the training set of the other profile. In particular, activities and relations in the model that are also frequent in the log are highlighted in dark blue. Activities that are present in the log but missing in the model are represented as dashed loops.

The two process models reveal that all students, independently from their performances, do not execute a structured and fixed process to develop the software artifact. In particular, the resulting processes are characterized by a quite high cyclomatic complexity with few main decision points at which several activities can be executed. However, from the structure point of view, it is evident that there are four main clusters of activities that are very correlated and are executed in a mutually exclusive fashion. These clusters correspond to the major IDE functions: (*i*) global commands that alter the behavior of the IDE and to switch working context; (*ii*) editor commands related to code writing; (*iii*) refactoring commands used to alter the entire software artifact by applying code generation and transformations rules; and (*iv*) debugging commands used to test the behavior of the source code and to find bugs.

As shown in the figures, interactions always start and return on the global commands that have, for this reason, the highest relative frequencies. From these commands, the interactions usually move towards other clusters. It is interesting to note that not all profiles exercise all clusters of activity. Looking at low-performance participants, we can see that most of the interactions happen in the editor cluster and the most frequent sequence is related to context switch from one source code file to another (this is confirmed by the fact that SwitchFileContext activity is present in almost all process cases). This means that low-performance participants tend to use the IDE more like a text editor ignoring the development-oriented features available (for example, they almost ignore refactoring and have

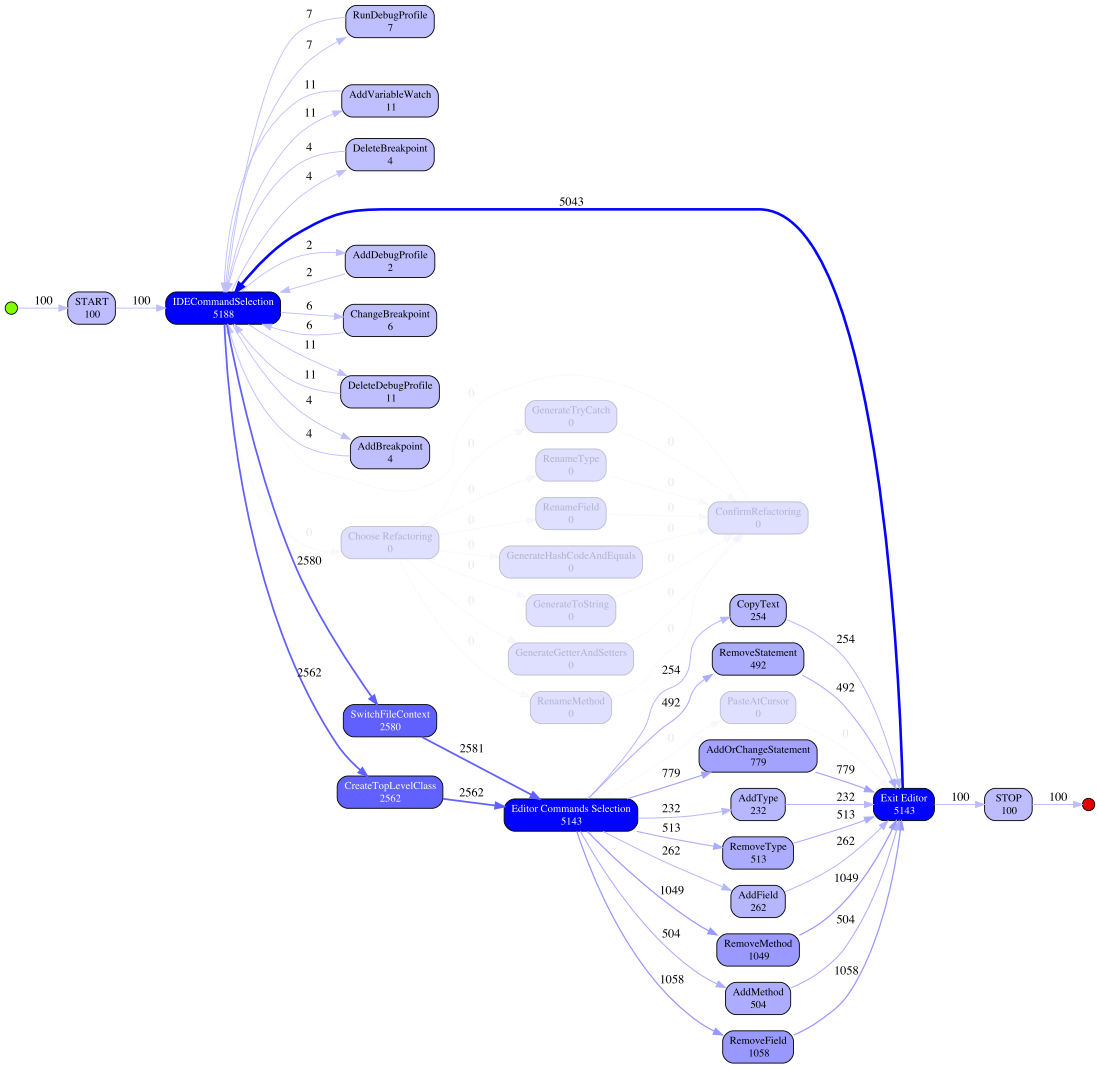

Fig. 4. Process model discovered using the DFG miner for high-performing developers.

a negligible usage of debugging commands). This is subverted by the high-performing participants: the sequences of activities pass through all the four clusters and this confirms that high-performance participants exploits all the IDE functionalities.

Comparing the processes in Fig. 4 and Fig. 5, we can observe that the development process for students with "High" profile includes a higher number of activities (31 different activities are performed) with respect to students with "Low" profile (in this case only 18 different activities are performed). The figures highlight that students with low profile tend to repeat a smaller number of activities and exhibit a higher frequency of remove commands (i.e., RemoveField, RemoveMethod, RemoveStatements, AddorChangeStatements).

Fig. 7 shows the deviances of a specific low-profile student with respect to the process mined from high-profile students. The differences are similar to the ones shown in Fig. 4. However, for this specific case, together with the refactoring commands, also most of the debugging commands are missing. Similarly, Fig. 6 describes the deviances of a specific high-profile student with respect to the mined process for students with low performance. This specific student still performs more activities (dashed loops) than the low-profile students but less than the ones identified in the general case in Fig. 5.

Fig. 10 reports the boxplot of the PMP of the traces in the validation set from the low-profile students with respect to the model representing the high-profile students (left-hand side of the figure), and the boxplot of the PMP of the traces in the validation set from the high-profile students with respect to the model representing the low-profile students (right-hand side of the figure). The figure shows that both the reference models are able to discriminate low- and high-profile students. However, when the high-profile model is used as a reference, the mean PMP for high and low-profile students are quite different (respectively 0.65 and 0.42). Similarly, when the reference model is the low-profile model high and low-profile students have different PMP distributions (the men PMP is respectively 0.41 and 0.7).

We evaluated the ability of classifying the students in the

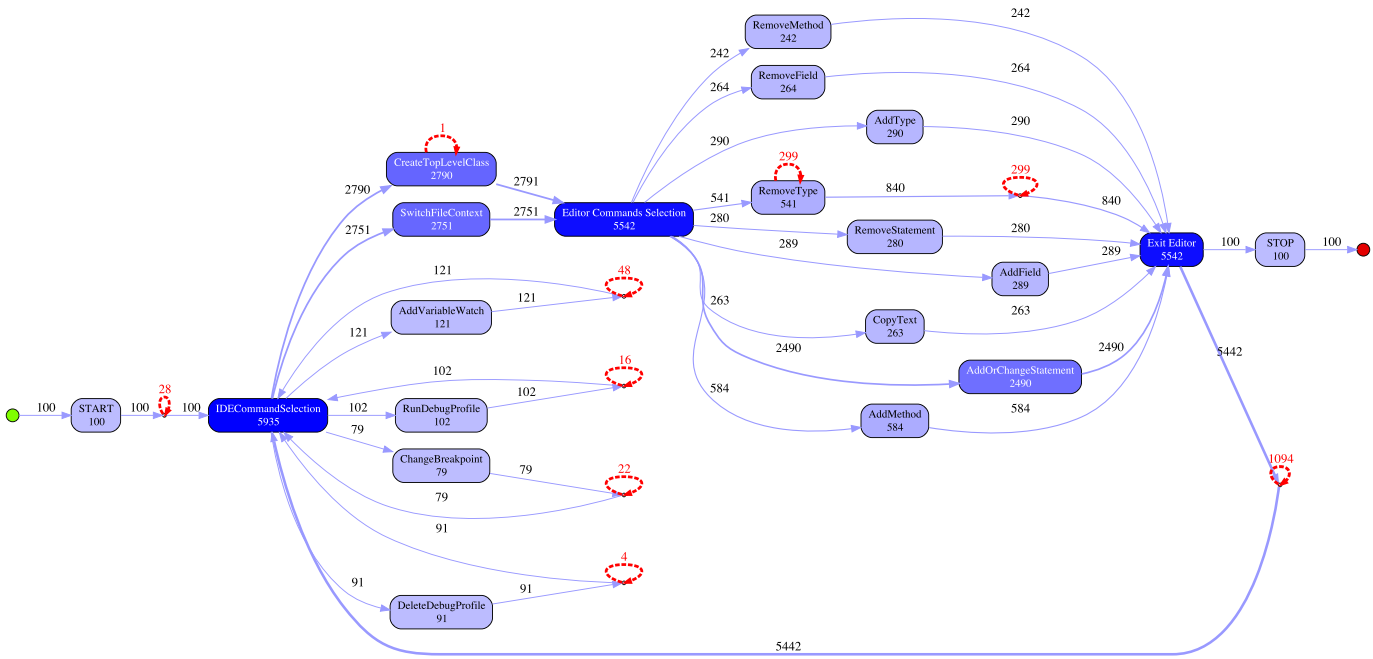

Fig. 5. Process model discovered using the DFG miner for low-performing developers.

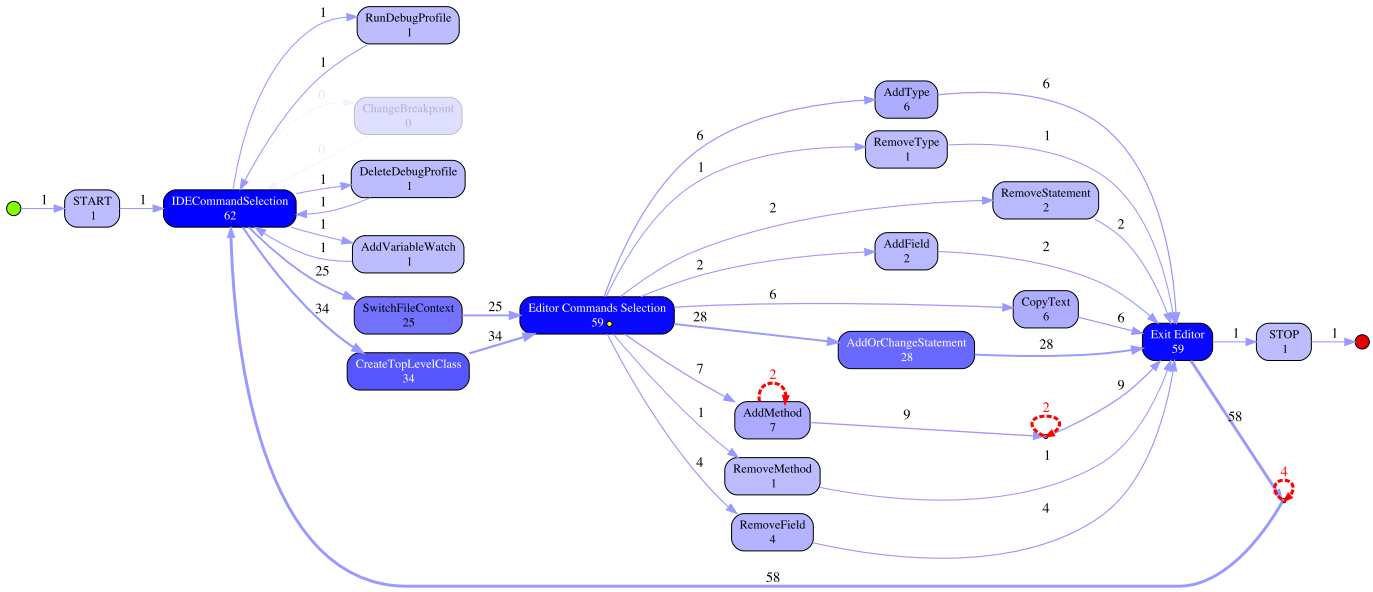

Fig. 6. Comparison of a high-profile student with the process model discovered using the DFG miner for low-performing developers.

union of the two test sets (low and high) by using the following measures:

1) *precision*,
2) *recall*,
3) *F-Measure*.

These measures are defined with respect to a *gold standard* that indicates the correct labeling of each trace. In our experiments, we assume that the gold standard is the evaluation of the teacher. Given the gold standard, we classify the trace related to a new developer into four categories: *i*) true-positive (T_P: the teacher evaluates the student as high (respectively low) and the PMP with respect to the model representing the high-profile (respectively low-profile) students is above a fixed threshold); *ii*) false-positive (F_P: the teacher evaluates the student as low (respectively high) and the PMP with respect to the model representing the high-profile (respectively low-profile) students is above a fixed threshold); *iii*) true-negative (T_N: the teacher evaluates the student as low (respectively high) and the PMP with respect to the model representing the high-profile (respectively low-profile) students is below a fixed

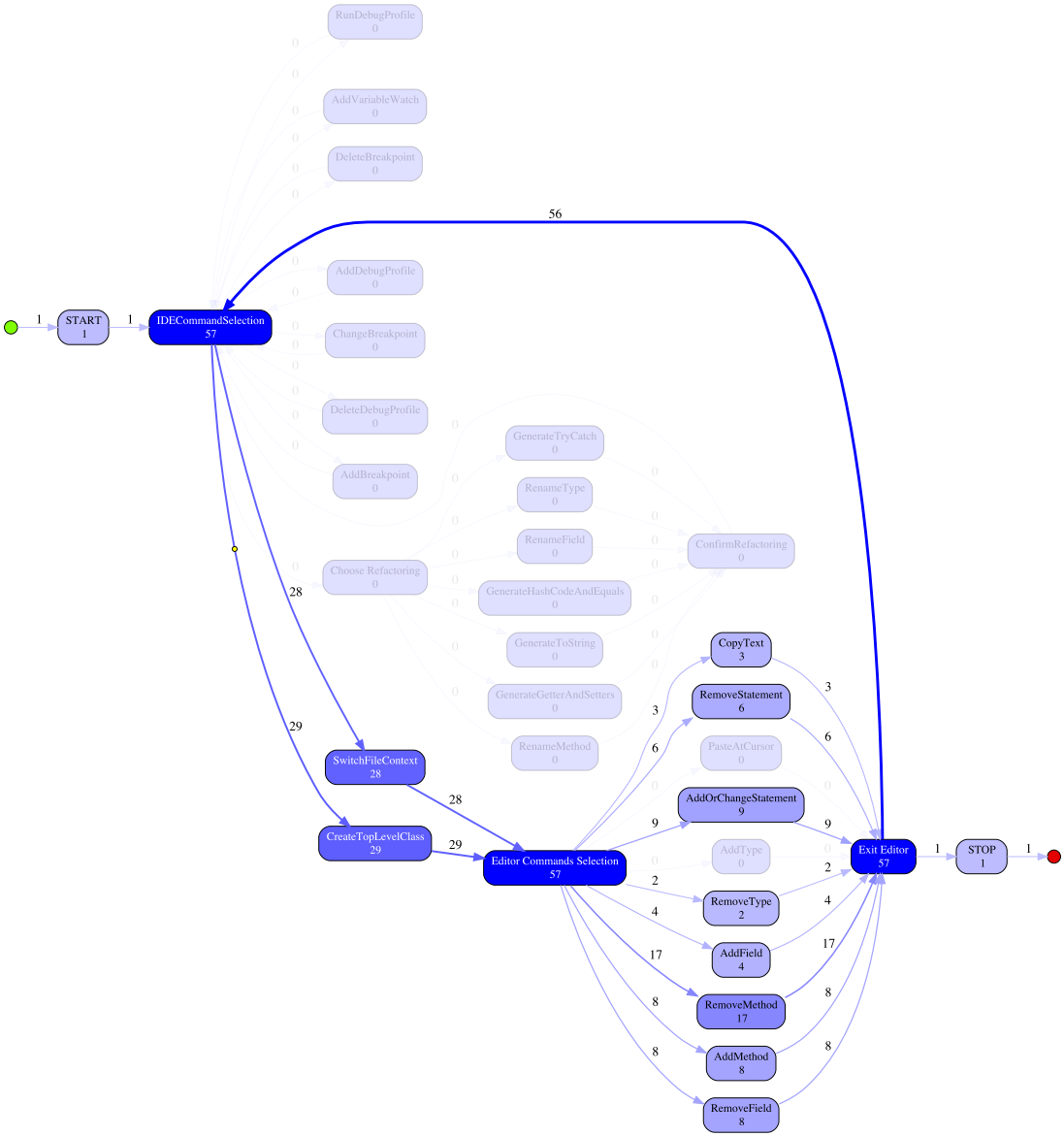

Fig. 7. Comparison of a low-profile student with the process model discovered using the DFG miner for high-performing developers.

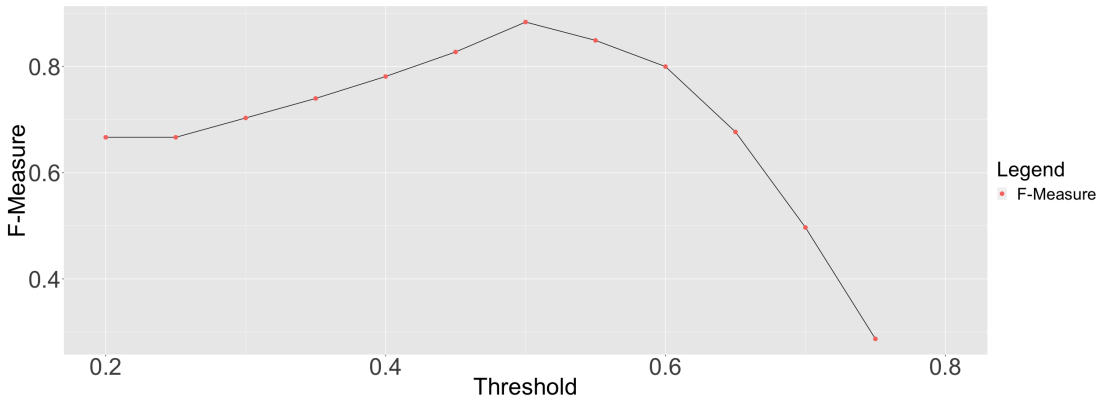

Fig. 8. F-Measure trend of High model with respect to the threshold: the best value is obtained at 0.52 for validation set.

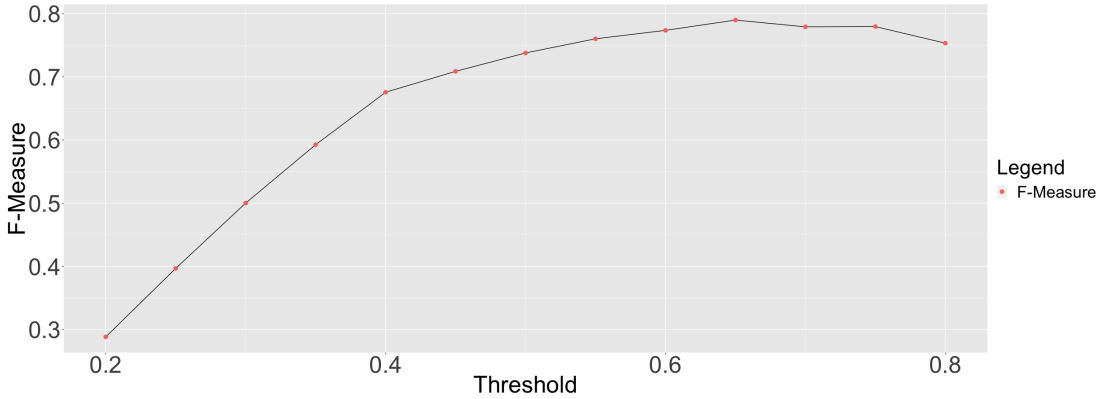

Fig. 9. F-Measure trend of Low model with respect to the threshold: the best value is obtained at 0.65 for validation set.

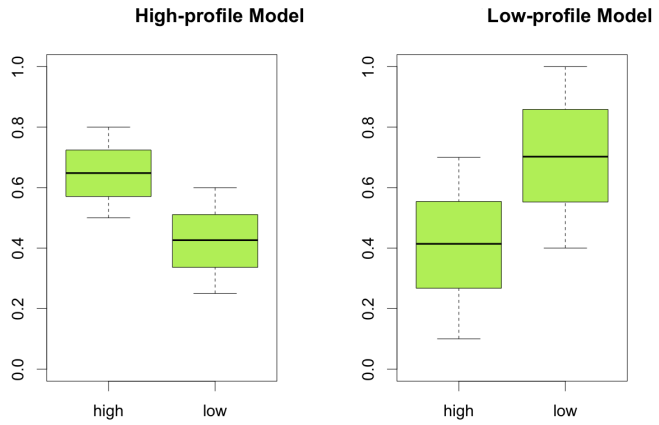

Fig. 10. Boxplots of the PMP distribution using both high and low-profile model on 20 logs (validation set).

threshold); *iv*) false-negative (F_N: the teacher evaluates the student as high (respectively low) and the PMP with respect to the model representing the high-profile (respectively low-profile) students is below a fixed). Precision and recall are then defined as:

$$Precision = \frac{T_P}{T_P + F_P} \qquad Recall = \frac{T_P}{T_P + F_N}. \tag{1}$$

While F-Measure is the harmonic mean of precision and recall:

$$FMeasure = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}. \tag{2}$$

Fig. 8 shows the F-Measure computed using the model extracted by high-profile students and the union of the 2 validation sets for different precision thresholds. The optimal threshold is chosen as the one that guarantees the highest F-Measure (in this case 0.52). Using the model extracted by high-profile students and the union of the two test sets (i.e., including both high and low profiles), we evaluated if the identified precision threshold was able to discriminate between high- and low- profile students in the test set. For this experiment, we obtained precision, recall and F-Measure of, respectively, 0.86, 0.85 and 0.85.

Fig. 9 shows the F-Measure computed using the model extracted by low-profile students and the union of the two validation sets for different precision thresholds. The optimal threshold is chosen as the one that guarantees the highest F-Measure (in this case 0.65). Using this threshold, the ability of classifying the students in the union of the two test sets (low and high) was evaluated by computing precision, recall and F-Measure that are, respectively, 0.88, 0.71 and 0.71.

We can conclude by saying that the proposed approach provides feedback to the users by graphically representing the deviances between coding behaviors coming from developers with different skills and performance. The approach allows developers to evaluate their coding behavior and the comparison with the high-performant developers drives the low-performant developers to improve the way they code. In addition, the approach is able to classify new behaviors by looking at the precision computed by comparing the new behavior with the reference models representing how high- and low-profile developers face software development.

V. Related Work

The application of process mining to the study of software development processes has been already investigated in the literature [2], [**?**], [17]. This interest is due to the availability of software repositories that allow deriving information about software development processes that can be analyzed through process mining approaches.

Several process mining techniques have been recently used to mine development processes starting from logs coming from version management systems or from document management systems. In [17], the authors describe a process mining framework used to obtain software process models useful for the analysis and optimization of software development processes. Other studies ([15], [18], [2], [1]) highlight the benefits of software process modeling with the aid of software repositories. Some of these works have been focused on examining bug reports for detecting defect lifecycles [2], [1] or data coming from document management systems for analyzing requirement engineering processes [15], [18]. In [6], recorded behavior from processes in the context of software

engineering is analyzed using process mining. Process mining techniques are also used to perform conformance checking to compare software development processes. For example, in [13], the authors apply a Markov chain based process mining technique to analyze software development processes. The studies presented above are mainly focused on the whole software development lifecycle. In this paper, we show how conformance checking can be used to analyze event logs generated from IDE usage and classify coding behaviors. A similar approach was reported in [5]. This paper, however, only describes the main idea leaving its development and validation to future work.

VI. Conclusions and Future work

In this paper, we presented an environment to capture event logs collected when developers use the Eclipse IDE and perform conformance checking of these logs with respect to a reference model. The approach was validated in a real context consisting of a group of 40 students of an advanced Java course involved in a set of programming tasks. Results demonstrate that the adopted process mining techniques are useful in understanding if the developers' behavior during the development sessions is compliant with a reference model and have the potential to give feedback useful to improve the coding process. In the future, we can use this environment to evaluate the impact of different code programming aspects (i.e., adopted languages, development principles, design patterns and practices) on the way the developers solve a given problem.

References

[1] Bug report networks: varieties, strategies, and impacts in a f/oss development community. *IET Conference Proceedings*, pages 80–84(4), January 2004.

[2] Mario Luca Bernardi, Marta Cimitile, Damiano Distante, Fabio Martinelli, and Francesco Mercaldo. Dynamic malware detection and phylogeny analysis using process mining. *International Journal of Information Security*, Jun 2018.

[3] R. Brooks. Towards a theory of the cognitive processes in computer programming. *Int. J. Hum.-Comput. Stud.*, 51(2):197–211, August 1999.

[4] J. C. A. M. Buijs, B. F. van Dongen, and W. M. P. van der Aalst. A genetic algorithm for discovering process trees. In *2012 IEEE Congress on Evolutionary Computation*, pages 1–8, June 2012.

[5] J. Caldeira and F. B. e. Abreu. Software development process mining: Discovery, conformance checking and enhancement. In *2016 10th International Conference on the Quality of Information and Communications Technology (QUATIC)*, pages 254–259, Sep. 2016.

[6] Jonathan E. Cook and Alexander L. Wolf. Discovering models of software processes from event-based data. *ACM Trans. Softw. Eng. Methodol.*, 7(3):215–249, July 1998.

[7] T. DeMarco and T. Lister. *Peopleware: Productive Projects and Teams*. Pearson Education, 2013.

[8] R. Guindon and B. Curtis. Control of cognitive processes during software design: What tools are needed? In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, CHI '88, pages 263–268, New York, NY, USA, 1988. ACM.

[9] Christian W. Günther and Wil M. P. van der Aalst. Fuzzy mining – adaptive process simplification based on multi-perspective metrics. In Gustavo Alonso, Peter Dadam, and Michael Rosemann, editors, *Business Process Management*, pages 328–343, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.

[10] M. Leemans and W. M. van der Aalst. Process mining in software systems: Discovering real-life business transactions and process models from distributed systems. In *2015 ACM/IEEE 18th International Conference on Model Driven Engineering Languages and Systems (MODELS)*, volume 00, pages 44–53, Sept. 2015.

[11] Sander J. Leemans, Dirk Fahland, and Wil M. Aalst. Scalable process discovery and conformance checking. *Softw. Syst. Model.*, 17(2):599–631, May 2018.

[12] Sander J. J. Leemans, Dirk Fahland, and Wil M. P. van der Aalst. Discovering block-structured process models from event logs - a constructive approach. In José-Manuel Colom and Jörg Desel, editors, *Application and Theory of Petri Nets and Concurrency*, pages 311–329, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

[13] Artini M. Lemos, Caio C. Sabino, Ricardo Massa Ferreira Lima, and César Augusto Lins Oliveira. Conformance checking of software development processes through process mining. In *SEKE*, 2011.

[14] Cong Liu, Boudewijn F. van Dongen, Nour Assy, and Wil M. P. van der Aalst. Component behavior discovery from software execution data. In *2016 IEEE Symposium Series on Computational Intelligence, SSCI 2016, Athens, Greece, December 6-9, 2016*, pages 1–8, 2016.

[15] Audris Mockus, Roy T. Fielding, and James D. Herbsleb. Two case studies of open source software development: Apache and mozilla. *ACM Trans. Softw. Eng. Methodol.*, 11(3):309–346, July 2002.

[16] Nancy Pennington, Adrienne Y. Lee, and Bob Rehder. Cognitive activities and levels of abstraction in procedural and object-oriented design. *Human-Computer Interaction*, 10(2-3):171–226, 1995.

[17] Vladimir Rubin, Christian W. Günther, Wil M. P. van der Aalst, Ekkart Kindler, Boudewijn F. van Dongen, and Wilhelm Schäfer. Process mining framework for software processes. In Qing Wang, Dietmar Pfahl, and David M. Raffo, editors, *Software Process Dynamics and Agility*, pages 169–181, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg.

[18] Walt Scacchi. Understanding requirements for open source software. In Kalle Lyytinen, Pericles Loucopoulos, John Mylopoulos, and Bill Robinson, editors, *Design Requirements Engineering: A Ten-Year Perspective*, pages 467–494, Berlin, Heidelberg, 2009. Springer Berlin Heidelberg.

[19] Wil M. P. van der Aalst. Big software on the run: in vivo software analytics based on process mining (keynote). In *ICSSP*, pages 1–5. ACM, 2015.